

Weather App

Technical Walkthrough Video — Script / Template

Target length: about 6–7 minutes (the brief says 5–8 minutes, and longer than 8 will not be reviewed). This is a guide, not a script to read word for word — the brief says the narration must be mostly your own words, so say each point in your own way.

Before recording: run the app once so the demo goes smoothly. Do not show `Secrets.h` on screen (it has your API key). Have `ofApp.cpp` open in Visual Studio. Speak slowly, and scroll to each function as you talk about it — that is what fills the time.

0:00 – 0:30 — Intro

Show: Face the app / title screen

- This is my Weather App for CodeLab II. It is built in C++ using openFrameworks, and it uses a free website called WeatherAPI to get the current weather for any place you type in.
- I will show it running first, then explain how the code works.

0:30 – 1:15 — Demo

Show: The running app

- Search "London" — point out temperature, condition, feels-like, humidity and wind.
- Search an empty box — it shows "Type a location first".
- Search a made-up place — it shows "Location not found".
- So it handles a normal search, an empty search and a wrong search without crashing.

1:15 – 2:00 — Project layout and design choice

Show: Solution Explorer, then ofApp.h

- The project is small on purpose. Almost all the code is in one file, `ofApp.cpp`, with `ofApp.h` next to it. I am still learning C++, so one place I can read from top to bottom was easier for me than many files.
- In `ofApp.h` I keep the things the app needs to remember: the text the user typed, the message shown on screen, and the weather values like temperature and humidity.
- There is also a separate file called `Secrets.h` that holds my API key. It is not uploaded to GitHub, so my key stays private. I only show a template version, `Secrets.example.h`.

2:00 – 2:45 — Starting up

Show: `setup()`

- `setup()` runs once when the app opens. It sets the window title and background colour, and loads two fonts — a normal one and a big one for the place name.
- The most important line is `ofRegisterURLNotification`. It tells openFrameworks: when a reply from the internet arrives, call my function `urlResponse`.
- It also sets where the search box and the Search button sit, as two rectangles, so I can draw them and check mouse clicks later.

2:45 – 3:45 — Sending the request

Show: searchWeather()

- When you press Enter or click Search, this function runs. First it checks the box is not empty.
- Then it builds a web address using the place you typed and my API key, and sends it with `ofLoadURLAsync`.
- The important word is "Async". It means the request runs in the background, so the window does not freeze while it waits for the internet.
- Spaces in the place name are swapped for `%20`, so a name like New York works inside a web address.

3:45 – 5:00 — Reading the reply

Show: urlResponse()

- `openFrameworks` calls this function by itself when the reply comes back.
- First it checks the status code. 200 means success. 400 means the place was not found, 401 means the key is wrong, and anything else shows a general error message.
- If it worked, the reply comes as JSON, which is just text organised with labels. I read it with `ofJson` and pick out the temperature, condition, humidity and wind.
- I use `.value` with a fallback for each field. So if a field is ever missing, the app uses a safe default instead of crashing.
- I also wrapped the reading in try and catch, so a badly formed reply shows an error message instead of closing the app.

5:00 – 5:45 — Keyboard and mouse

Show: keyPressed() and mousePressed()

- `keyPressed` handles typing. Enter starts the search, Backspace deletes a letter, and normal letters and numbers are added to the text.
- I only allow normal keys, and I stop at 60 characters, so strange keys or a huge input cannot cause problems.
- `mousePressed` checks if the click was inside the Search button rectangle, using the `.inside` function, and if so it runs the same `searchWeather` function.

5:45 – 6:30 — Drawing the screen

Show: draw()

- `draw()` only shows what is stored at that moment: the search box, the Search button, the status message, and the weather details if there is a result.
- The status message is red for errors and grey otherwise, using one true-or-false variable.
- `draw()` does no network work at all. That separation made each part easier to write and to test.

6:30 – 7:00 — Improvements and close

Show: Face the camera

- With more time I would add a weather icon, and remember recent searches.
- I kept this app simple so I understand every line, which mattered more to me than adding extras I could not explain.

- That is my Weather App — thanks for watching.

Reminders

- Explain how the code works, not how to use the app — the brief marks the technical explanation.
- Say each point in your own words — do not read this word for word.
- 5–8 minutes total. Check the length before uploading; over 8 minutes will not be reviewed.
- Upload through Panopto, not as a normal file attachment.